started, it moves into the *running* state, where it actively executes code. The `docker run` command we've seen creates a new container, then starts it running.

In the running state, a container can be restarted, stopped, killed, or paused. Pausing a container suspends the running processes so that they can be resumed some time later. Stopping a container attempts to shut down gracefully by sending a terminate signal (`SIGTERM`) to the main process inside the container—falling back to a forceful shutdown with a kill signal (`SIGKILL`) if this fails. Killing a container jumps straight to the forceful termination.

A container moves into the *stopped* state if it is stopped, or killed, or if the main process running inside it terminates. The stopped state is similar to the created state: the container sits there doing nothing until it is called upon. From there, the container can either be restarted or, if you have no more use for it, removed from the system. With that in mind, let's see how this works in practice using Compose.

First of all, let's check what containers are currently running. To do this, we use the `ps` command:

```
$ docker-compose ps
   Name              Command           State           Ports
------------------------------------------------------------------------
myapp_web_1   bin/rails s -b 0.0.0.0   Up      0.0.0.0:3000->3000/tcp
```

The listing includes the container name, the command used to start it, its current state, and its port mappings. Here you can see the container for our Rails server; it's still running from when we previously ran `docker-compose up -d` (`Up` means it's running).

If we now wanted to stop the Rails server, we'd do so with the `stop` command. By default, the command would apply to *all* services listed in our `docker-compose.yml` file. For example, to stop all containers in the entire application, we would run:

```
$ docker-compose stop
```

To target a *particular* service, we'd specify the service name after the action like so:

```
$ docker-compose stop <service_name>
```

This may seem like a moot point since, currently, `web` is the only service we have defined. However, we'll soon be adding more services, starting in Chapter 5, Beyond the App: Adding Redis, on page 59. It's common to want to target commands at a specific service, so it's very useful to remember this pattern—particularly as all the `docker-compose` commands follow it.